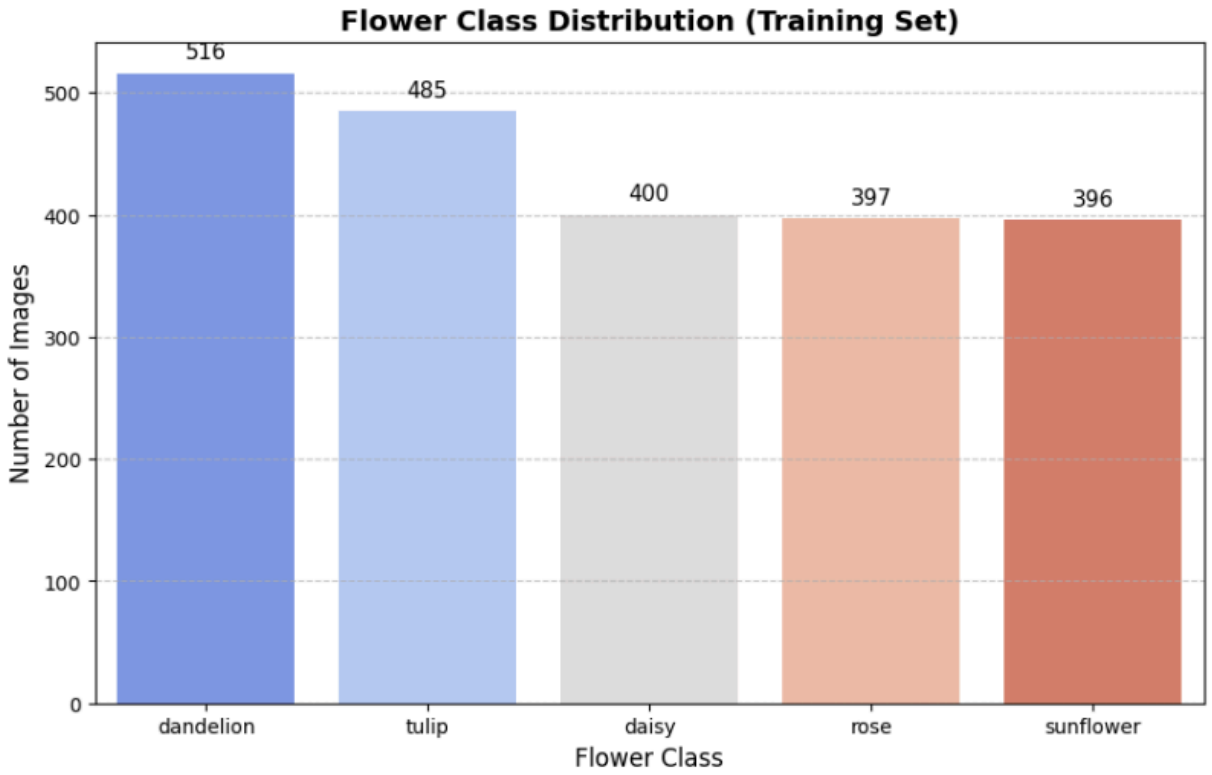
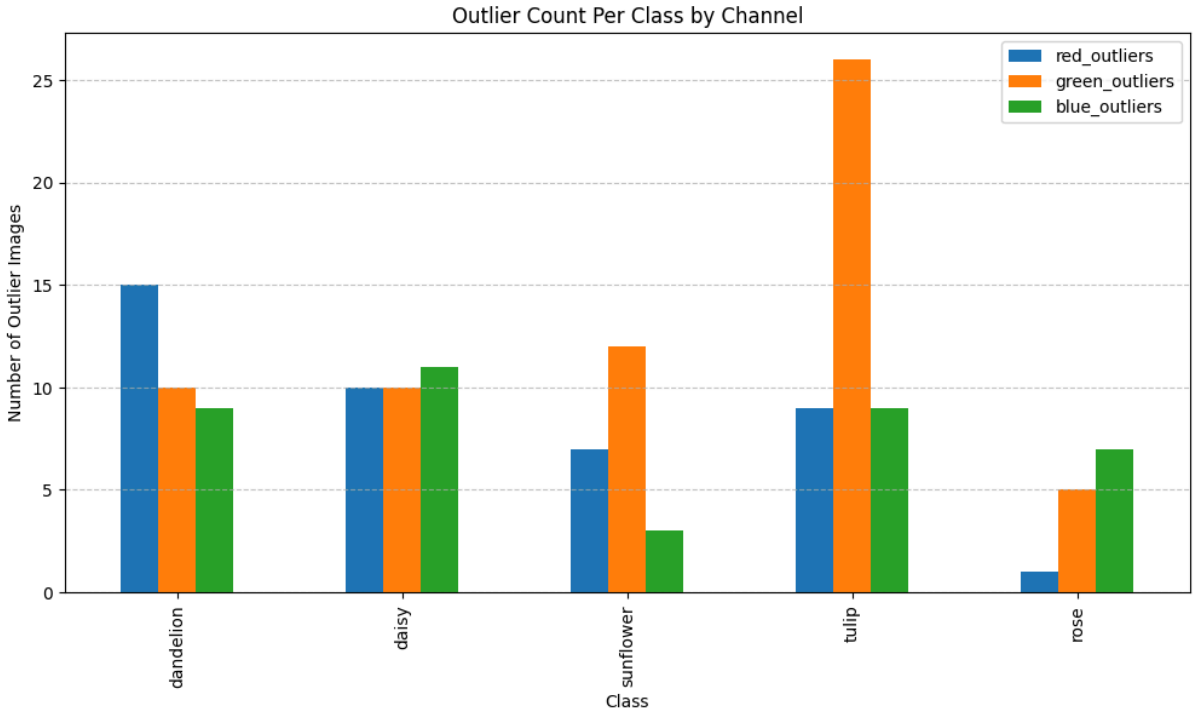
About the dataset

Dataset Size: The dataset contains **2746 images** of flowers.
Training dataset: 2746 images
Test dataset: 552 images
Type of Data: The dataset is composed of **JPEG images** of flowers. These are divided into five classes: **tulip, rose, sunflower, daisy and dandelion**.



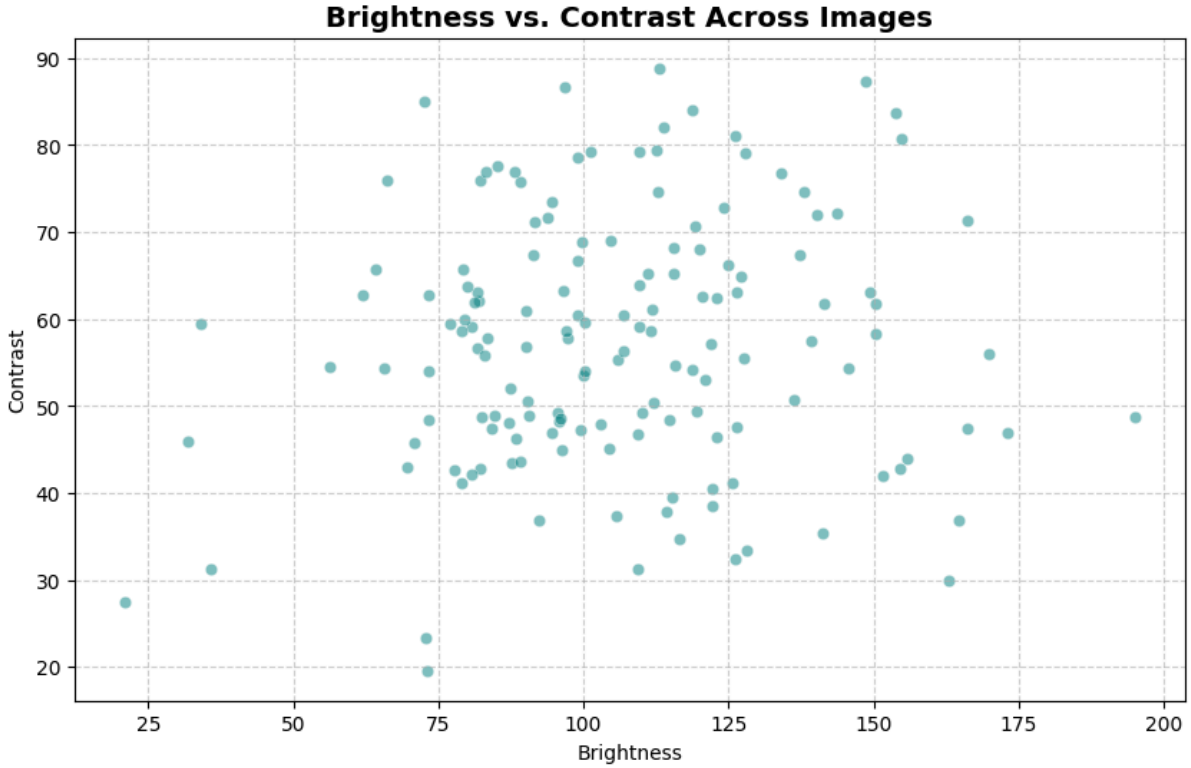
Outlier Detection: A boxplot visualization was used to identify outlier intensities within each RGB channel. Outliers are marked as individual points outside the whiskers, which represent 1.5 times the interquartile range (IQR). * Total images analyzed: 2,746 * Images with at least one outlier channel: 84 (3.06%) * Channel-specific outliers: Red: 31 images (1.13%) Green: 46 images (1.68%) Blue: 45 images (1.64%) These results indicate that outliers are relatively infrequent, impacting just over 3% of the dataset. The distribution across channels is fairly balanced, with a slightly higher incidence in the Green and Blue channels. Outlier images may require closer inspection or special preprocessing during downstream analysis.

Outlier Detection (Per Category/Class)



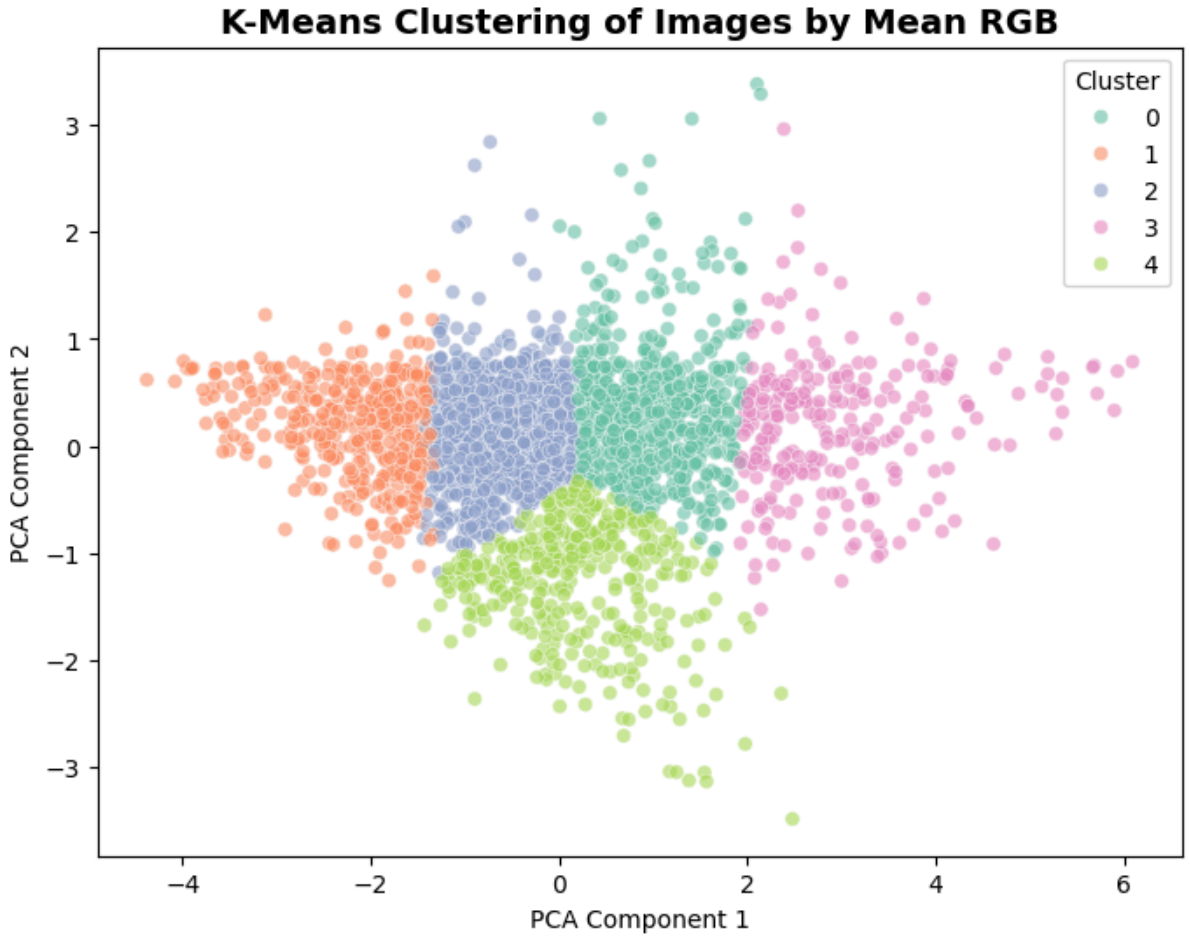
Outlier Analysis by Category: To assess potential anomalies in the dataset,we conducted an outlier analysis of mean RGB channel intensities for each image, segmented by flower class. Outliers were defined using the interquartile range (IQR) method, where any mean channel value falling outside 1.5 times the IQR from the first or third quartile for a given class and channel was considered an outlier. The results show that the dataset is generally well-balanced, with only a small number of outliers detected in each class. Tulip images exhibited slightly higher variability in the green channel, while rose samples were the most consistent across all channels. Overall, the dataset shows good color consistency, and only a few images may require further inspection or preprocessing before model training.

Image Brightness and Contrast Analysis



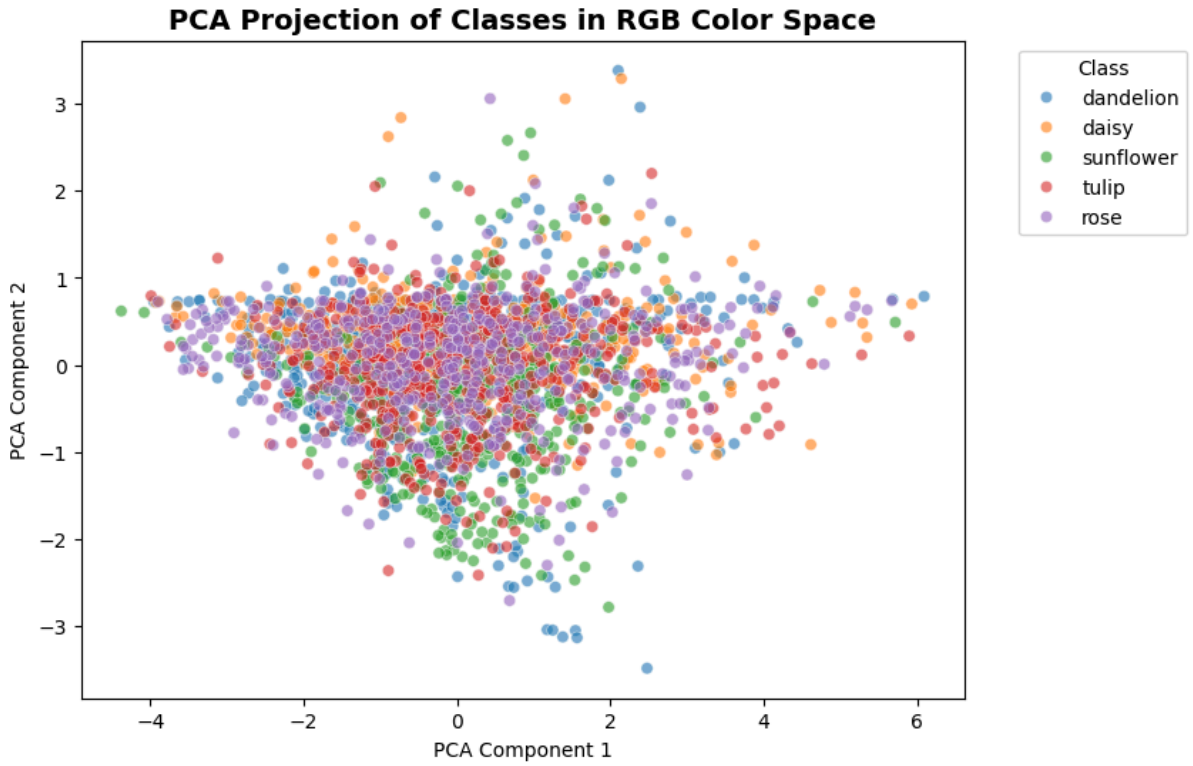
Brightness and Contrast Analysis: The distribution of brightness and contrast across images revealed substantial variability, indicating inconsistent illumination and tonal balance throughout the dataset. While most samples occupy a mid-range brightness (≈75–130) and contrast (≈40–70), several outliers exist, reflecting underexposed or overexposed conditions. This heterogeneity suggests that the dataset encompasses diverse environmental lighting and background conditions, which can enhance model robustness but also necessitates photometric normalization or adaptive histogram equalization to stabilize input intensity distributions before training.

K-Means Clustering of Image Color Profiles



K-Means Clustering of Images by Mean RGB: Clustering images in the normalized RGB color space using K-Means (k=5) exposed distinct groups reflecting dominant color palettes, confirming significant chromatic diversity across samples. The PCA-reduced visualization demonstrated clear cluster separations, implying meaningful color-based grouping patterns. This distribution highlights that mean RGB features capture broad spectral differences and can serve as low-level descriptors for pre-clustering or anomaly detection, though deeper spatial and contextual information remains necessary for fine-grained classification.

Per-Class Cluster Visualization



PCA Projection of Classes in RGB Color Space: The PCA projection of class labels in RGB space exhibited considerable overlap among categories, demonstrating that pure color features are insufficient for discriminating between classes such as daisy, rose, or tulip. This overlapping structure implies that the dataset's inter-class color similarity is high, and discriminative power must therefore arise from higher-order visual cues such as texture, petal morphology, and spatial gradients. Consequently, feature extraction via convolutional layers becomes essential to achieve class separability beyond chromatic attributes.

More EDA plots:

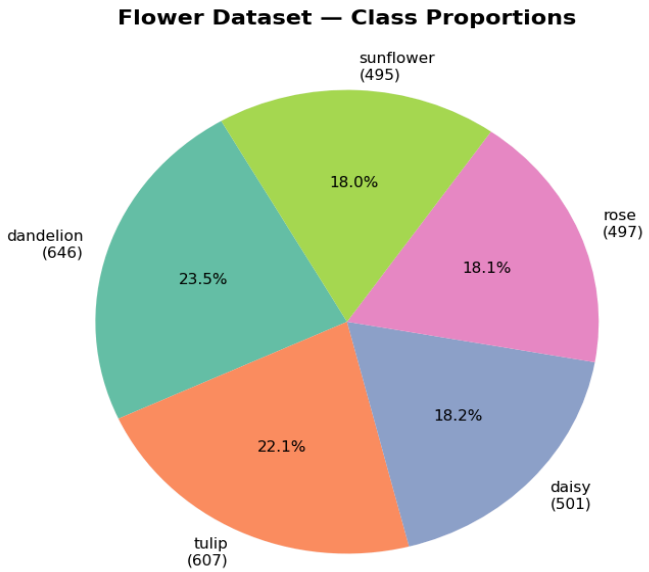
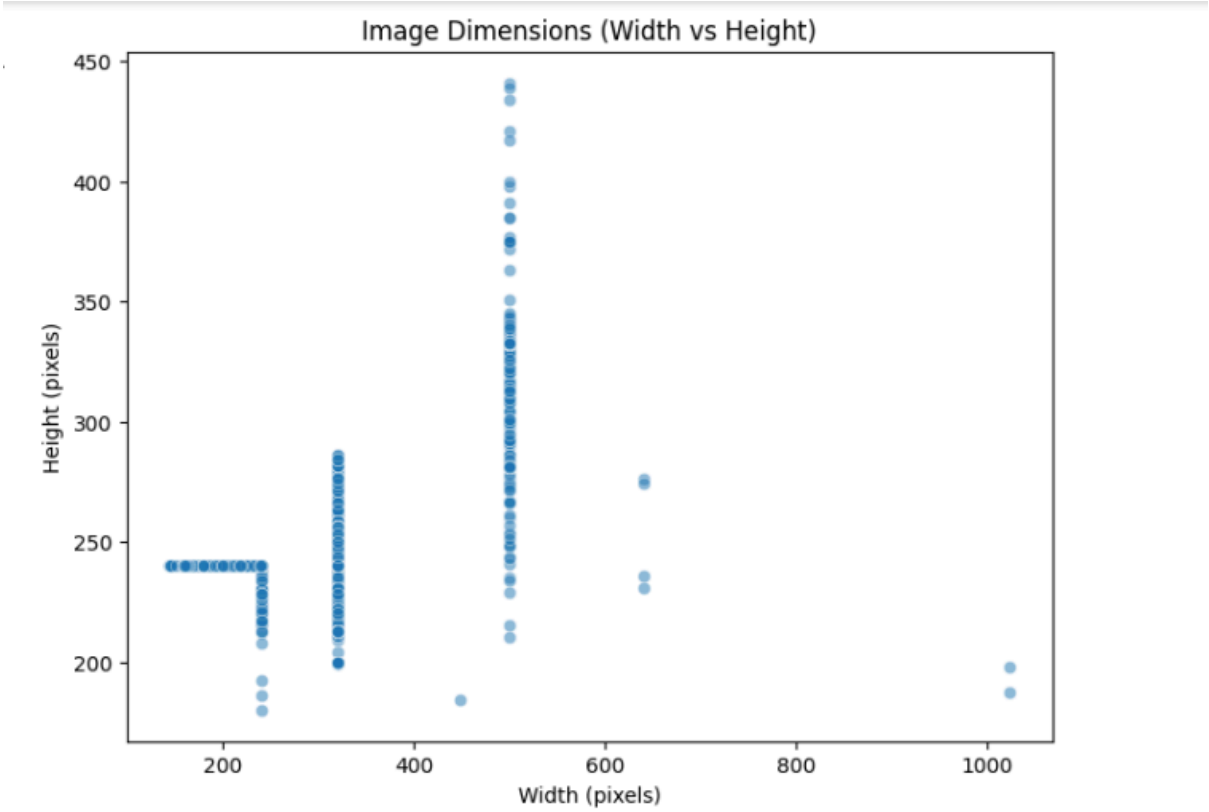


Image Dimension Analysis

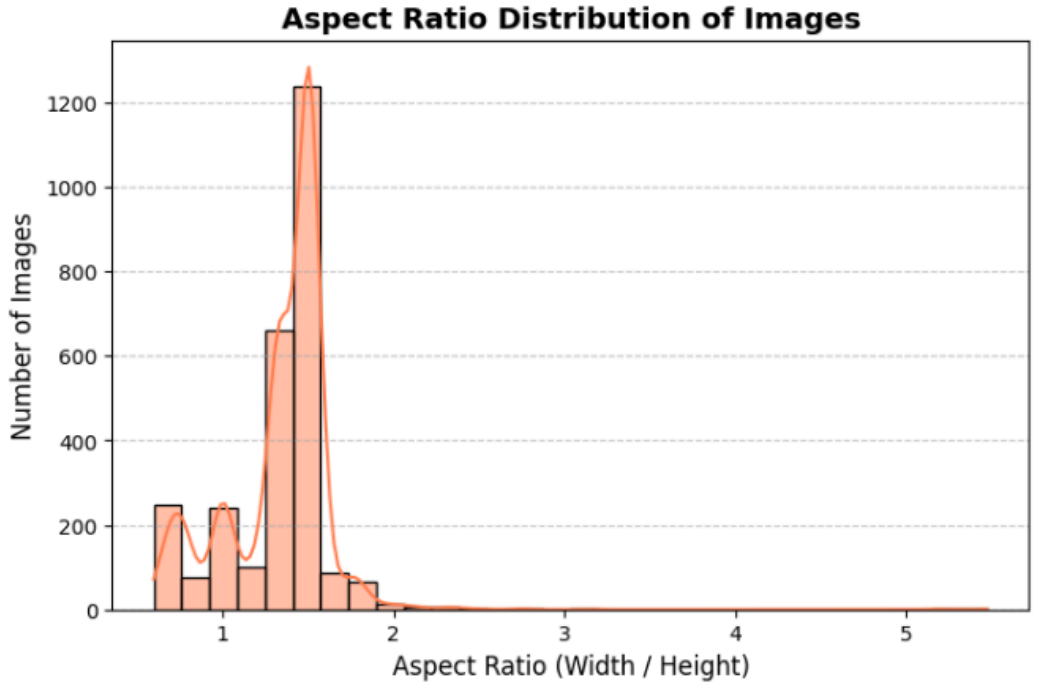


	Width	Height
count	2194.000000	2194.000000
mean	361.894257	267.670465
std	117.249905	47.914104
min	143.000000	180.000000
25%	320.000000	240.000000
50%	320.000000	240.000000
75%	500.000000	332.000000
max	1024.000000	441.000000

The scatter plot shows that image sizes in the dataset vary widely, indicating that the images are not uniform in resolution. The average width is around 362 px and height about 268 px, with sizes ranging from 143×159 to 1024×441 pixels. Several clusters in the plot suggest images come from different sources or preprocessing steps. Since CNN models require fixed input sizes, this variation means all images need to be resized to a standard dimension (e.g., 224×224 px) before training to ensure consistency and efficient processing.

ASPECT RATIO ANALYSIS

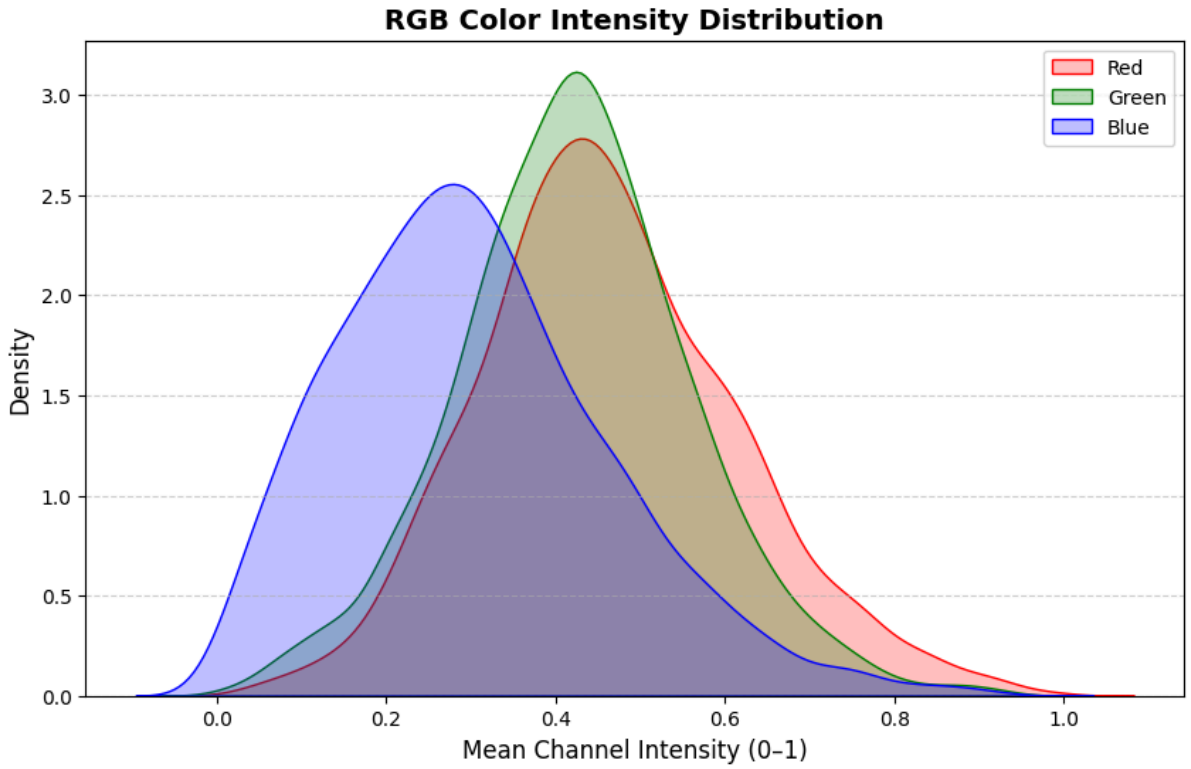
```
count    2746.000000
mean        1.335132
std         0.306930
min         0.595833
25%         1.250000
50%         1.457726
75%         1.502347
max         5.475936
Name: Aspect_Ratio, dtype: float64
```



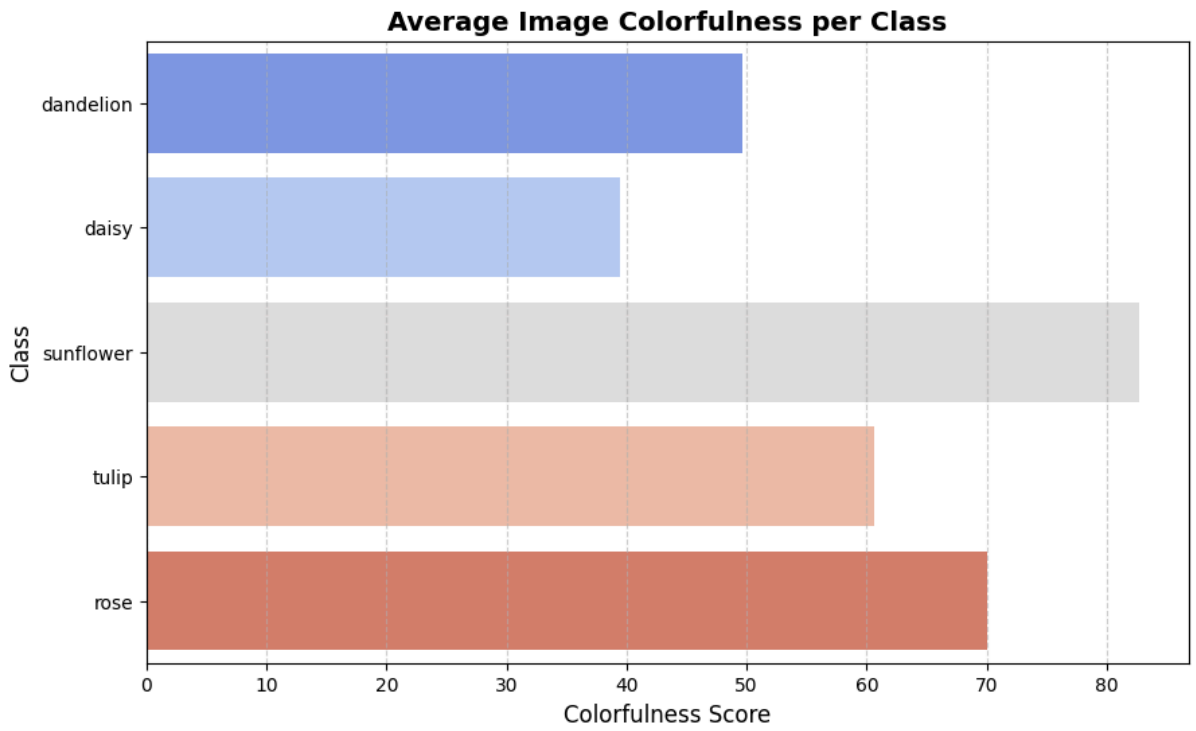

```
Average aspect ratio per class:
        Class  Aspect_Ratio
0      daisy      1.318337
1  dandelion      1.337294
2       rose      1.315831
3  sunflower      1.316668
4      tulip      1.377555
```

Aspect Ratio Analysis

The aspect ratio (width ÷ height) of all images was analyzed to understand the shape consistency across the dataset. The mean aspect ratio is approximately 1.33 with a standard deviation of 0.31, indicating that most images are slightly wider than tall. The majority of values fall between 1.25 and 1.50, showing moderate uniformity in image proportions. A few outliers with higher ratios suggest the presence of some non-standard or cropped images. Across classes, the average aspect ratio remains consistent (around 1.31–1.37), meaning no specific class shows strong deviation. Overall, the dataset maintains a fairly consistent image shape, but resizing all images to a fixed input size (e.g., 224×224 px) will still be necessary to ensure uniformity for CNN training.

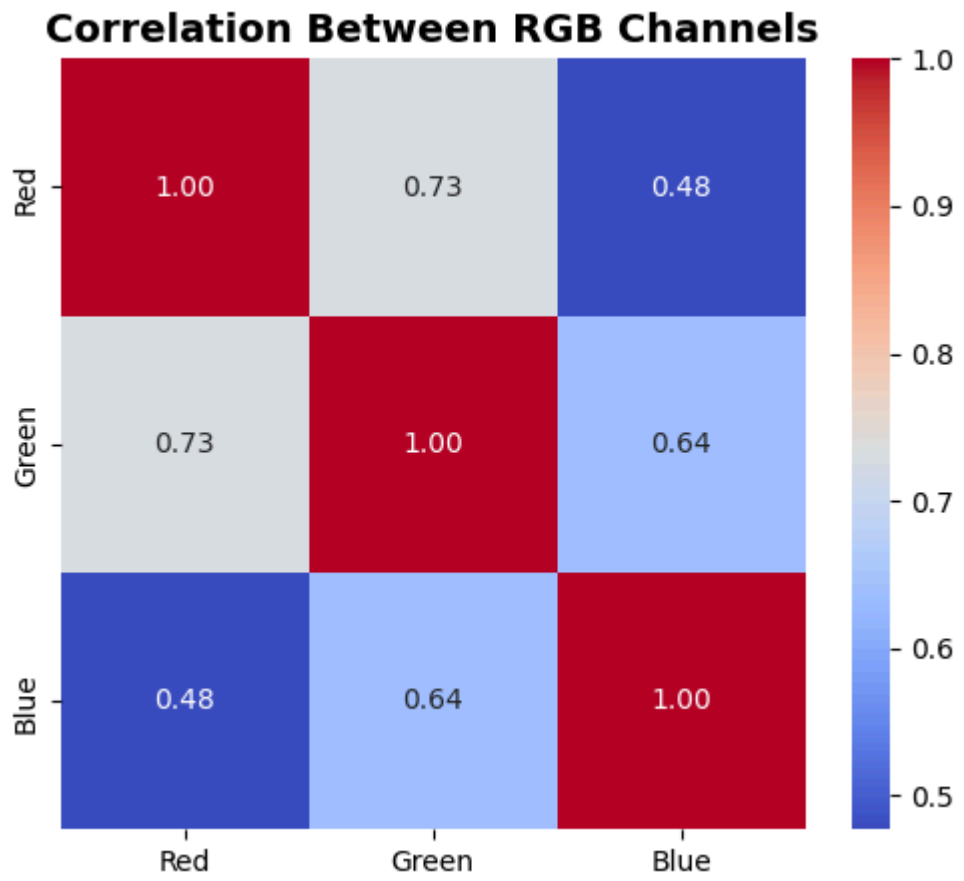


RGB Color Intensity Distribution : To better understand the overall color composition of the dataset, I computed the mean and standard deviation of each RGB channel across all training images. The color intensity distributions were visualized using kernel density estimation (KDE) plots. From the results, the red and green channels exhibit higher mean values (R=0.4623, G=0.4222) compared to the blue channel (B=0.3023), suggesting that the dataset is slightly biased toward warmer tones. The standard deviations (R=0.1541, G=0.1400, B=0.1617) indicate moderate variation in channel intensities, with blue showing the highest spread. This analysis provides useful insight for normalization, helping ensure consistent color balance during model training.

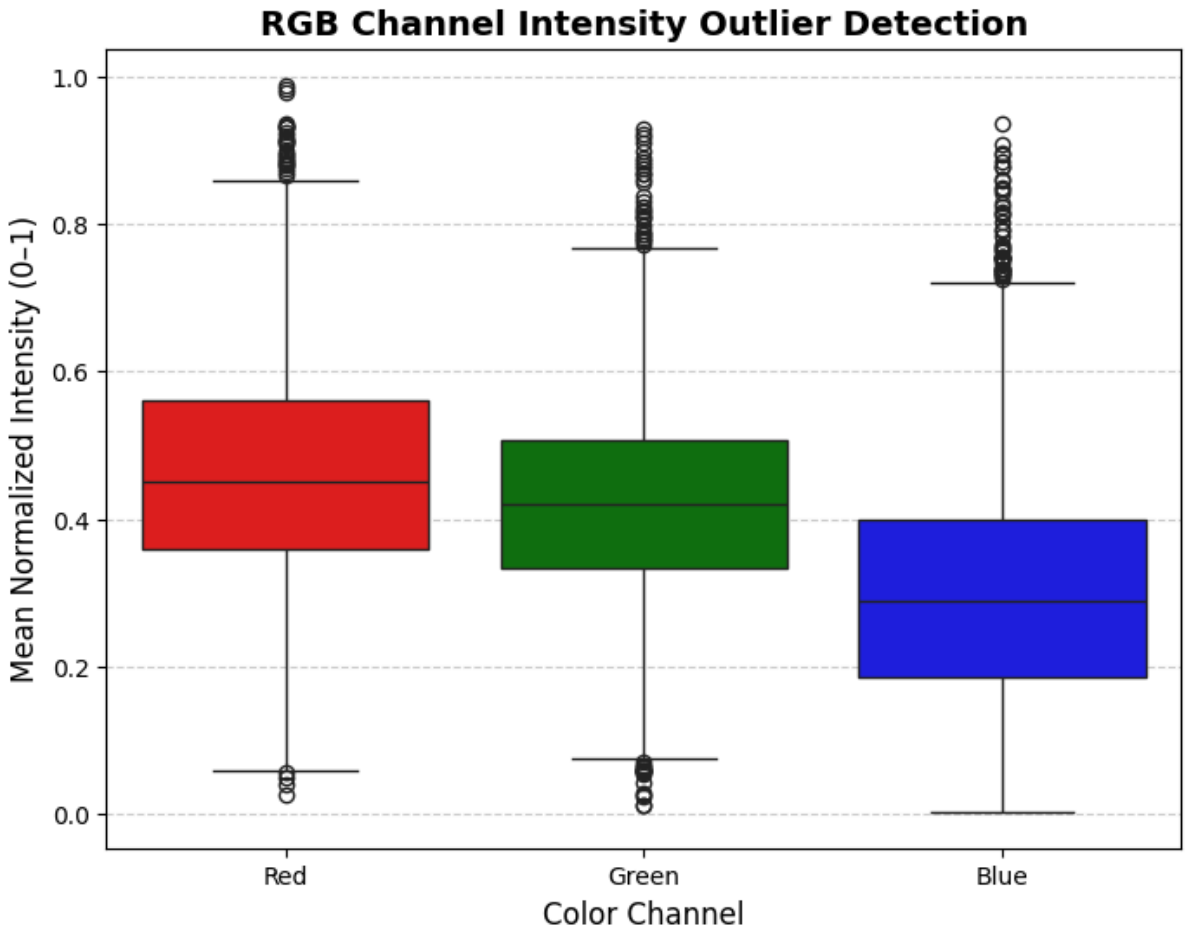


Average Image Colorfulness per Class

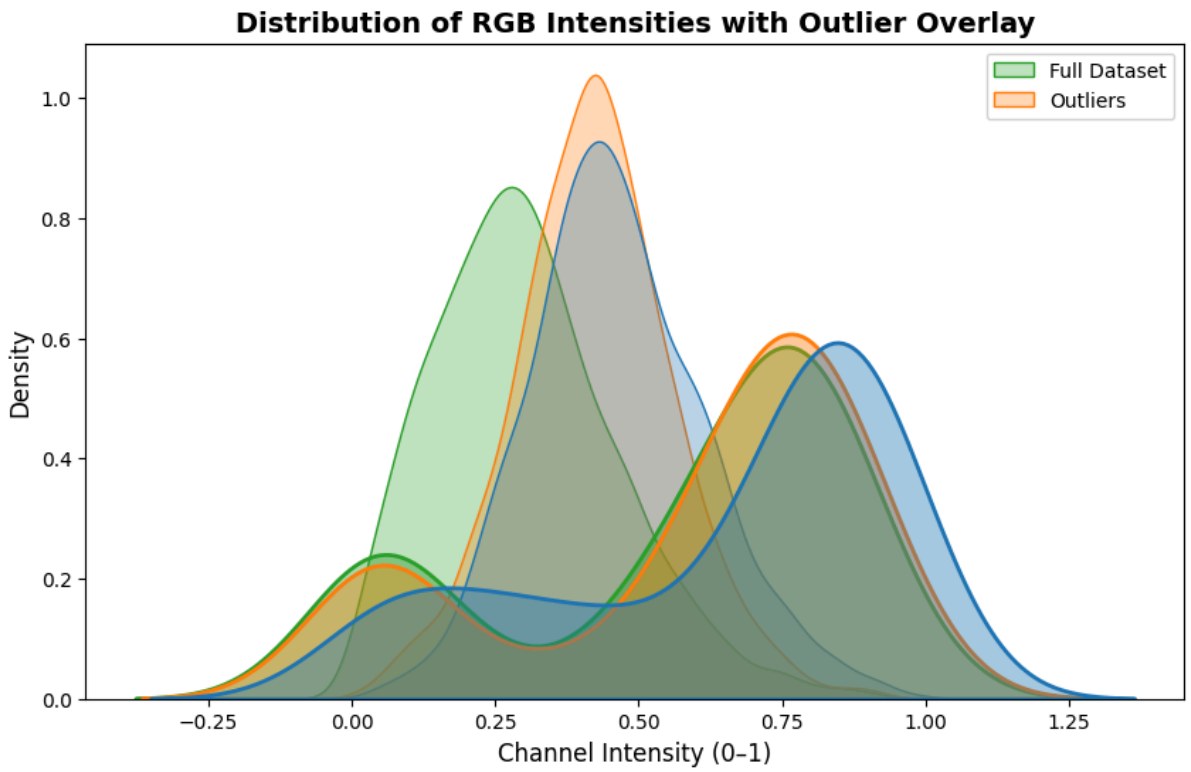
The colorfulness analysis shows that sunflowers and roses are the most vibrant classes, with the highest average colorfulness scores, indicating rich and saturated colors. Tulips also show moderate color variation, while dandelions and daisies are less colorful, reflecting their simpler color patterns. This suggests that different flower types vary in visual richness, which may affect how easily the model distinguishes them based on color cues.



The heatmap titled **"Correlation Between RGB Channels"** shows a strong positive correlation between the Red and Green channels (0.73), a moderate correlation between Green and Blue (0.64), and a weaker correlation between Red and Blue (0.48). This indicates that red and green intensities tend to vary together in most images—typical of natural scenes where these colors often coexist—while blue behaves more independently. Overall, the dataset's color channels are moderately interrelated, suggesting natural image content with distinct but overlapping spectral information across RGB components.



Distribution Visualization: Density plots compare the distribution of channel intensities for the full dataset against the subset of detected outlier values. Each channel shows a reasonably normal distribution, though the outlier overlay highlights slightly heavier tails, particularly in the Green and Blue channels. This suggests that, while most images have typical color intensities, a small subset exhibits atypical channel behavior.

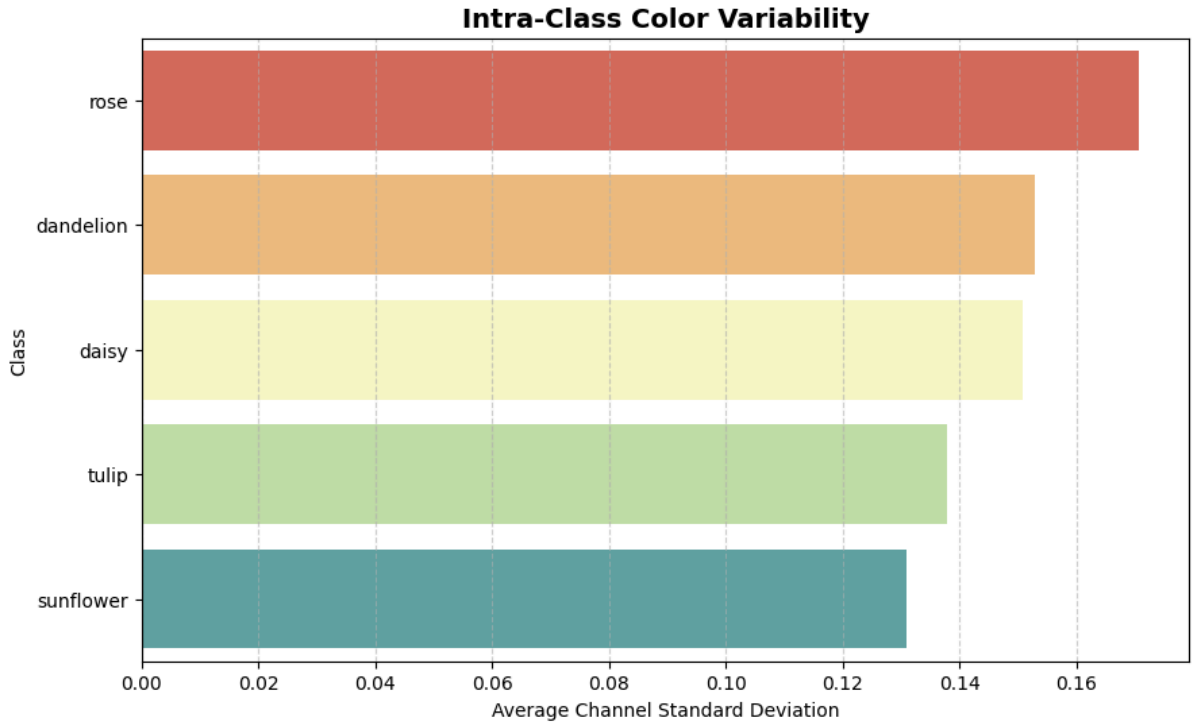


Outlier Detection: A boxplot visualization (Figure 2) was used to identify outlier intensities within each RGB channel. Outliers are marked as individual points outside the whiskers, which represent 1.5 times the interquartile range (IQR).

- Total images analyzed: 2,746
- Images with at least one outlier channel: 84 (3.06%)
- Channel-specific outliers:

Red: 31 images (1.13%) Green: 46 images (1.68%) Blue: 45 images (1.64%)

These results indicate that outliers are relatively infrequent, impacting just over 3% of the dataset. The distribution across channels is fairly balanced, with a slightly higher incidence in the Green and Blue channels. Outlier images may require closer inspection or special preprocessing during downstream analysis.



To further understand color consistency within each flower category, I calculated the average standard deviation of the RGB channel means per class as a measure of intra-class color variability. As shown in the horizontal bar plot, rose images exhibit the highest intra-class color variability, followed by dandelion and daisy, while sunflower images have the lowest variability among all classes. This suggests that rose images in the dataset tend to have a wider range of colors or lighting conditions, whereas sunflower images are more consistent in terms of color representation. Such variations could affect model performance and may require class-specific augmentation or normalization strategies.

About our model

Model Architecture

The model is a **VGG-16**, a well-known convolutional neural network architecture. we are using a version pre-trained on the ImageNet dataset.

The architecture was modified for specific task using a technique called **fine-tuning**:

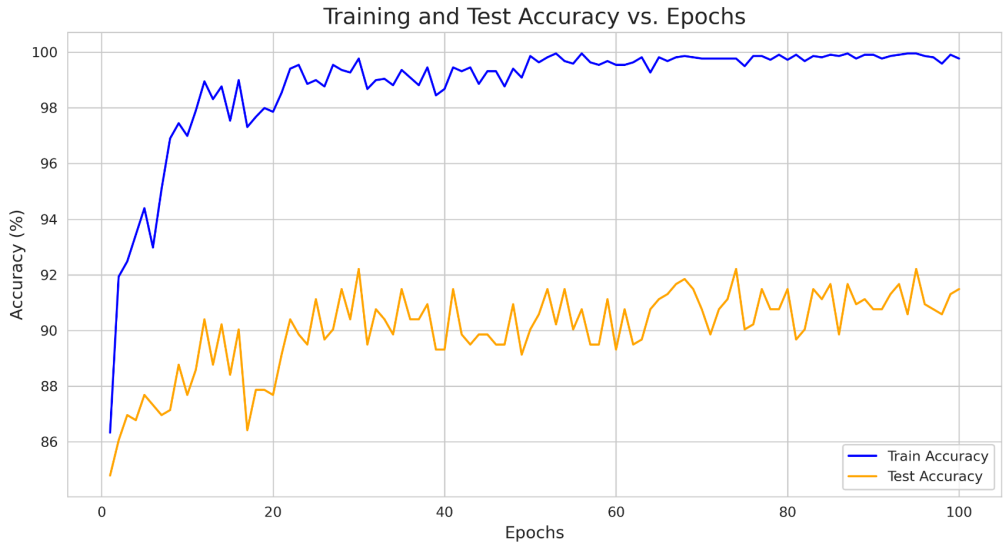
1. **Frozen Feature Extractor**: The weights of all the convolutional layers (`model.features`) were frozen. This means they were not updated during training, preserving the powerful feature-detection capabilities learned from ImageNet.
2. **Replaced Classifier**: The final fully connected layer (`model.classifier[6]`) was replaced with a new `torch.nn.Linear` layer. This new layer was specifically designed to output predictions for the number of classes present in flower dataset.

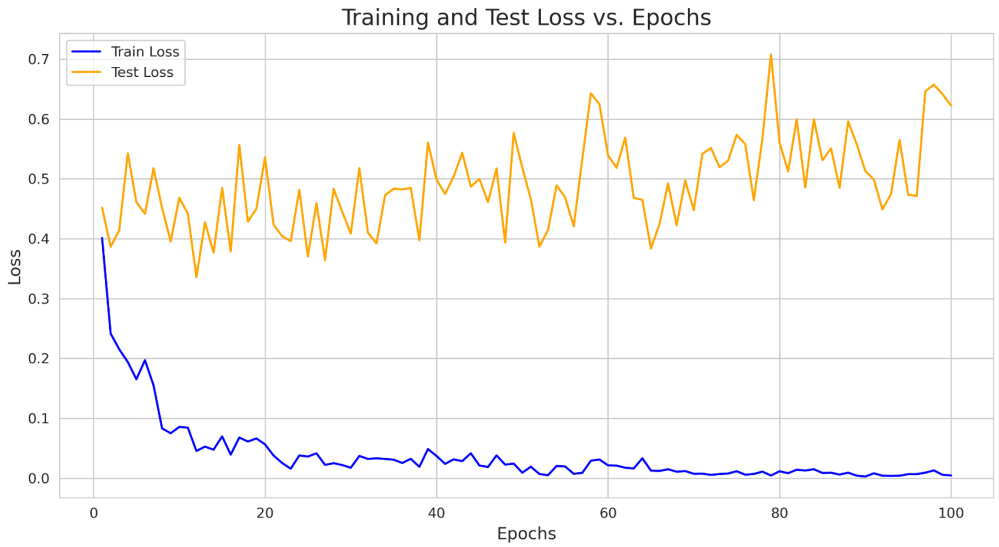
Essentially, we used the pre-trained VGG-16 as a fixed feature extractor and only trained the final classification layer to recognize our specific types of flowers.

Training Details

Based on our Python scripts and output plots, here are the key details of the training process:

- **Epochs**: The model was trained for **100 epochs**, as shown by the x-axis on accuracy and loss plots.
- **Optimizer**: The **Adam** optimizer was used. Notably, it was configured to only update the parameters of the new final classifier layer, which is consistent with the fine-tuning strategy.
- **Loss Function**: The **Cross-Entropy Loss** function (`nn.CrossEntropyLoss`) was used to measure the model's error.
- **Learning Rate**: **0.001** (from the default value in `finetune_vgg16.py`).
- **Batch Size**: **32** (from the default value in `finetune_vgg16.py`).
- **Training Time**: 85 minutes





About SHAP

We implemented SHAP on own as well referred to implementation

SHAP Library vs. "From Scratch" Implementation

we have two distinct scripts for SHAP analysis:

1. **`SHAP_analysis_SHAP_library.py`:** This script uses the established `shap` library to perform the analysis. It imports `shap` and uses `shap.KernelExplainer` to do the heavy lifting. This is the simpler, more direct method.
2. **`SHAP_analysis.py`:** This script does **not** import the `shap` library. Instead, it manually implements the core components of the KernelSHAP algorithm, including functions for generating masks (`generate_masks_random`), calculating the specific SHAP kernel weights (`kernel_shap_weight`), and solving the weighted linear regression (`solve_weighted_ridge`).

How the "From Scratch" Version Differs

our from-scratch implementation (`SHAP_analysis.py`) differs from the library version in how it executes the core steps of the KernelSHAP algorithm. The library abstracts these details away, while our script makes them explicit.

Feature	`SHAP_analysis_SHAP_library.py` (Library Version)	`SHAP_analysis.py` ("From Scratch" Version)

Core Logic	Relies on `shap.KernelExplainer` to manage the entire process.	Manually implements a weighted linear regression (`solve_weighted_ridge`) to find the feature importances (SHAP values).
Coalition Weighting	The specific kernel weighting formula, $\pi_x(z') = \frac{(M-1)}{(\binom{M}{	z'
Mask Generation	`explainer.shap_values` handles the sampling of different feature coalitions (superpixel masks) internally.	we created own function, `generate_masks_random`, to generate these coalitions.
Visualization	Can use built-in, powerful `shap` plotting functions like `image_plot` and `summary_plot`.	Requires custom plotting functions like `shap_to_heatmap` and `plot_shap_overlay` to visualize the results.

Pseudocode of "From Scratch" Implementation

Here is a pseudocode version of the logic in our `SHAP_analysis.py` file, which explains the steps taken to calculate SHAP values manually.

```
Function KernelSHAP_Image(model, image, num_samples):

  // 1. Segment the image into M superpixels
  segments, image_float = Superpixel_Segment(image)
  M = number of segments

  // 2. Get the model's prediction for the original, full image
  //    This determines the 'target_class' we want to explain.
  original_prediction = Predict(model, full_image)
  target_class = ArgMax(original_prediction)

  // 3. Generate coalitions (binary masks)
  //    Create 'num_samples' random masks, plus one empty and one full mask.
  masks_array = Generate_Masks(M, num_samples)

  // 4. Get model predictions for each coalition
  FOR each mask IN masks_array:
    masked_image = Create_Masked_Image(image_float, segments, mask)
    prediction = Predict(model, masked_image)
    // Store the logit (or probability) for the target_class
    model_outputs.append(prediction[target_class])
```

```
  ENDFOR

  // 5. Calculate SHAP kernel weights for each coalition
  FOR each mask in masks_array:
    coalition_size = Sum(mask) // Number of '1s' in the mask
    weights.append(Calculate_Kernel_Weight(M, coalition_size))
  ENDFOR

  // 6. Solve the weighted linear regression problem
  //    This is the core step that approximates the SHAP values.
  //    'y' is the vector of model_outputs.
  //    'X' is the matrix of binary masks.
  //    'W' is the vector of kernel weights.
  coefficients = Solve_Weighted_Linear_Regression(X, y, W)

  // The coefficients (excluding the intercept) are the SHAP values.
  shap_values = coefficients[1:]

  // 7. Return the results
  RETURN shap_values, segments

END Function
```

Explain our model

How SHAP Explains the Predictions

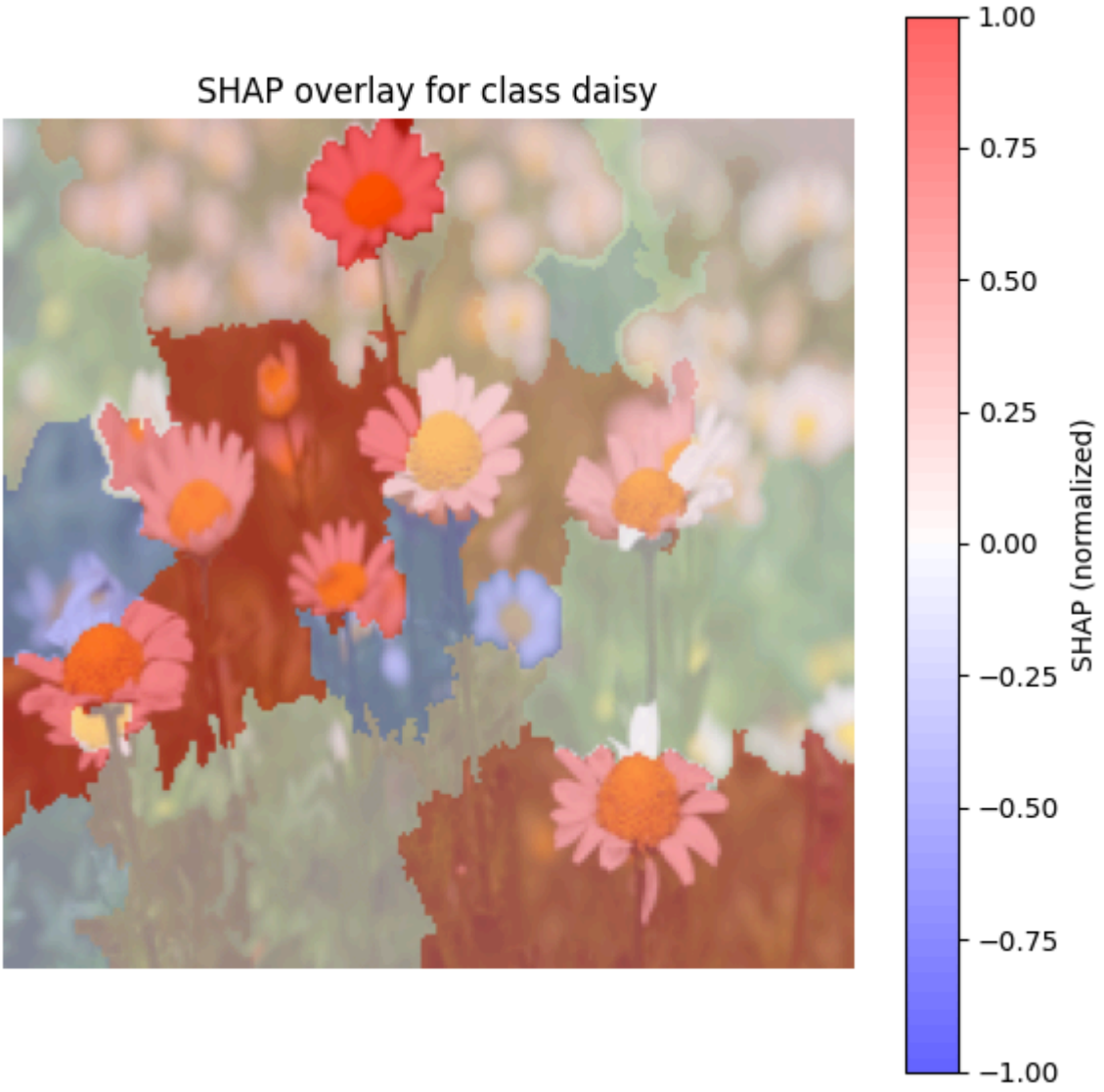
The key thing to understand from these plots is that **the red superpixels are the most important regions pushing the model to predict the correct class.**

- **Red Areas (Positive SHAP values):** These are patches of the image that provide strong **evidence FOR** the predicted class. The brighter the red, the more that patch contributed to the decision.
- **Blue Areas (Negative SHAP values):** These patches provide **evidence AGAINST** the prediction. They are features that push the model away from the final decision, perhaps because they resemble another flower or are just irrelevant background noise.
- **Transparent/Faded Areas (SHAP values near zero):** These patches had a negligible or neutral impact on the prediction.

Analysis of Each Flower

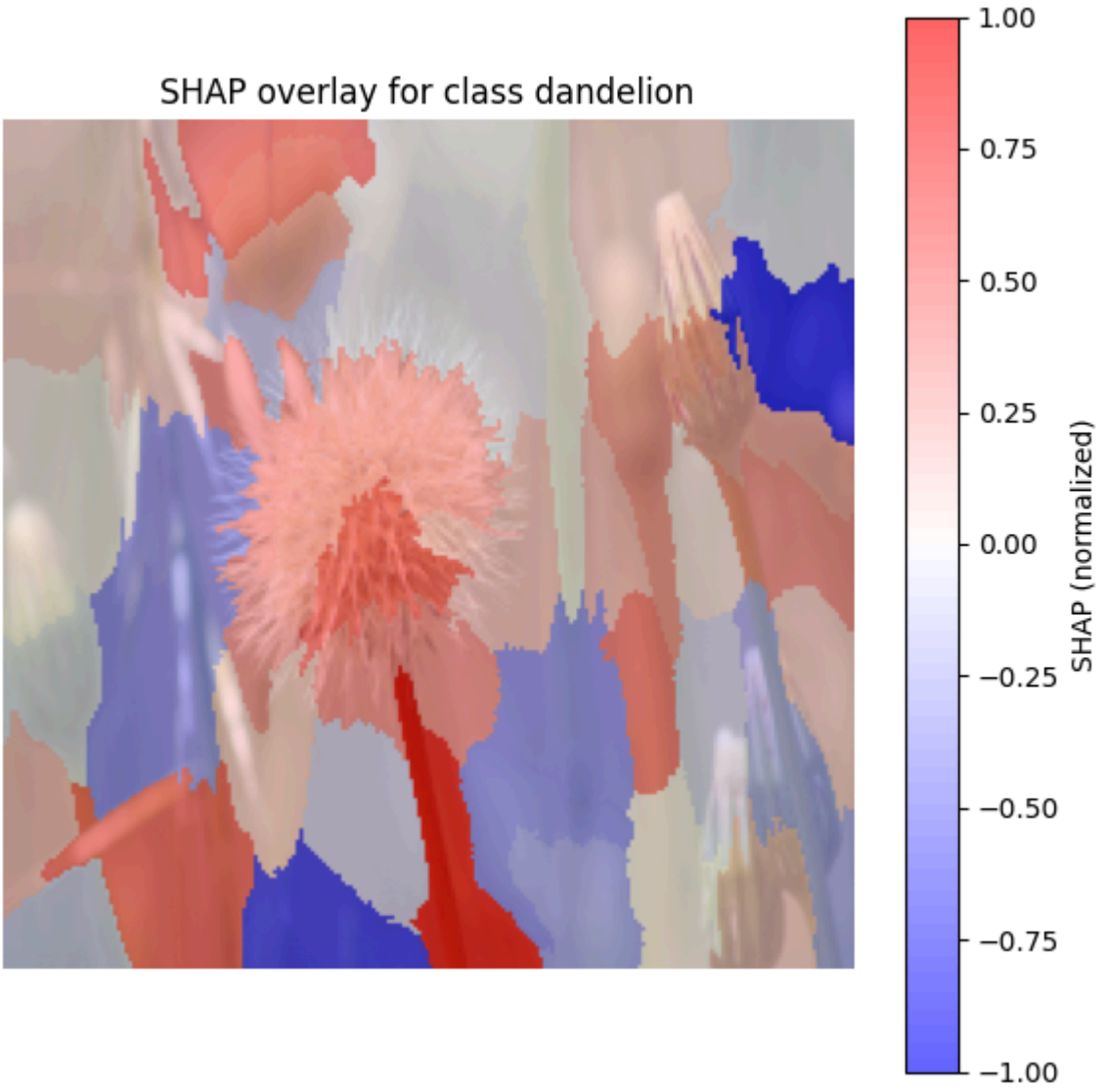
Here is a breakdown of what the model "saw" in each of our test images to arrive at its conclusion.

Daisy



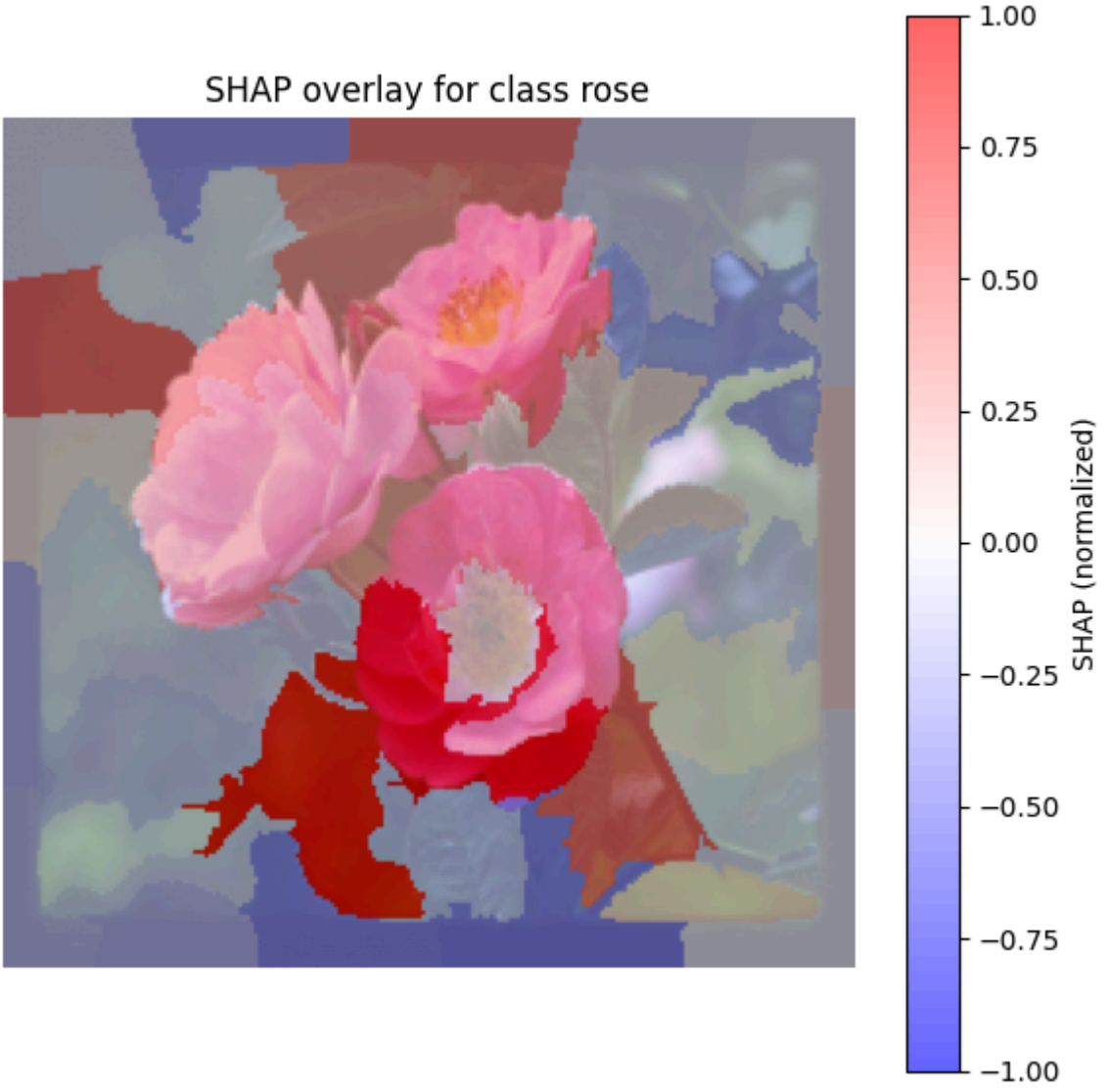
For the daisy image, the model's prediction was strongly influenced by the features most characteristic of a daisy. The most intense **red areas** are concentrated on the **distinct white petals** and the **bright yellow center (disc florets)** of the daisies in the foreground. This is an excellent sign, as it shows our model has correctly learned to identify the most important parts of a daisy. The **blue areas** are scattered in the background, likely on leaves or other flowers that are not daisies, providing counter-evidence that the model correctly ignored.

Dandelion



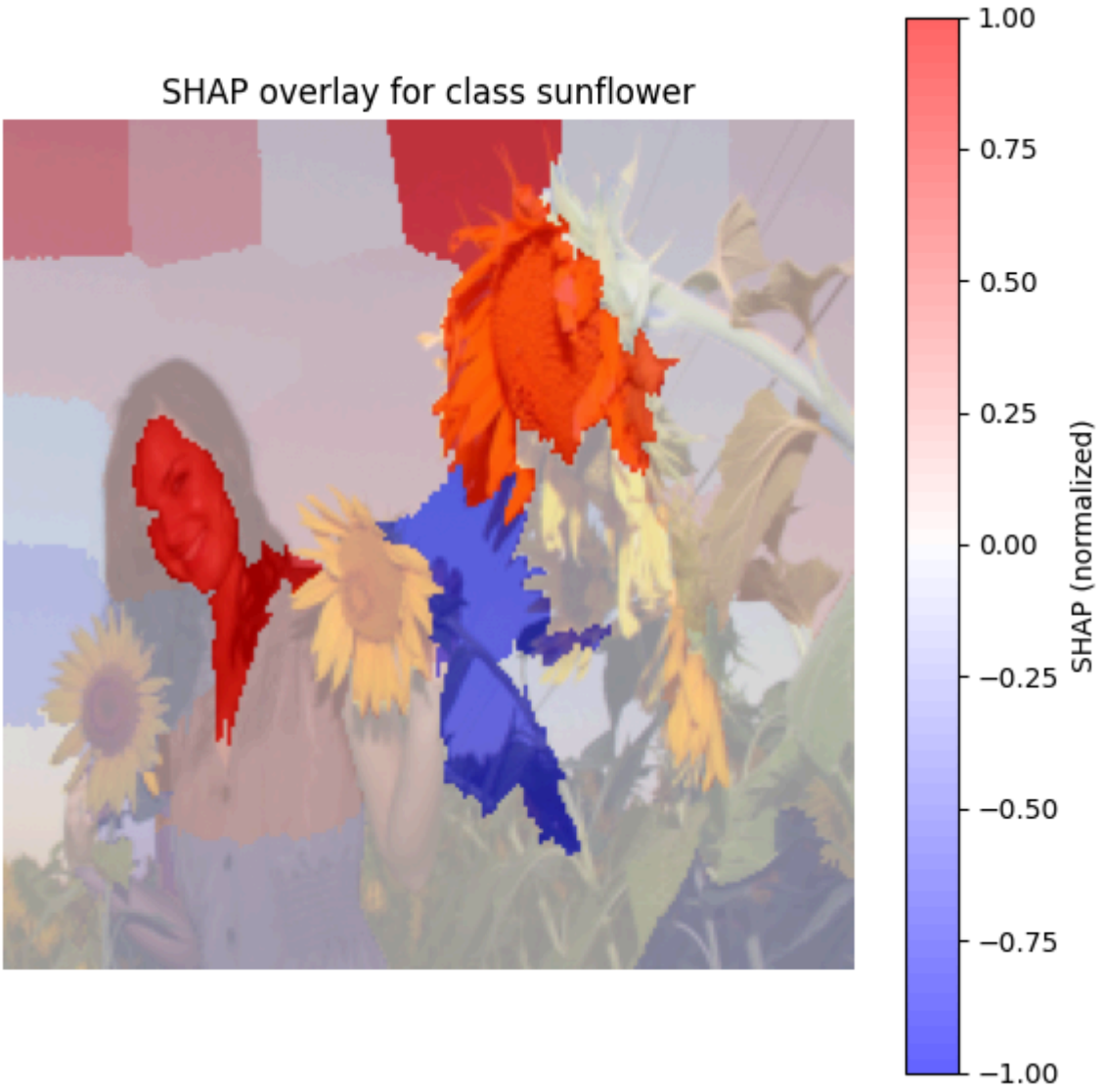
The model correctly identified the dandelion, with the **red areas** highlighting the unique, fluffy texture of the dandelion's **spherical seed head** (the "puffball") and parts of the **stem**. These features are the primary evidence for its prediction. The background foliage and other less-defined shapes are marked in blue, indicating they pushed against the "dandelion" classification, but their influence was weaker than the positive evidence from the flower itself.

Rose



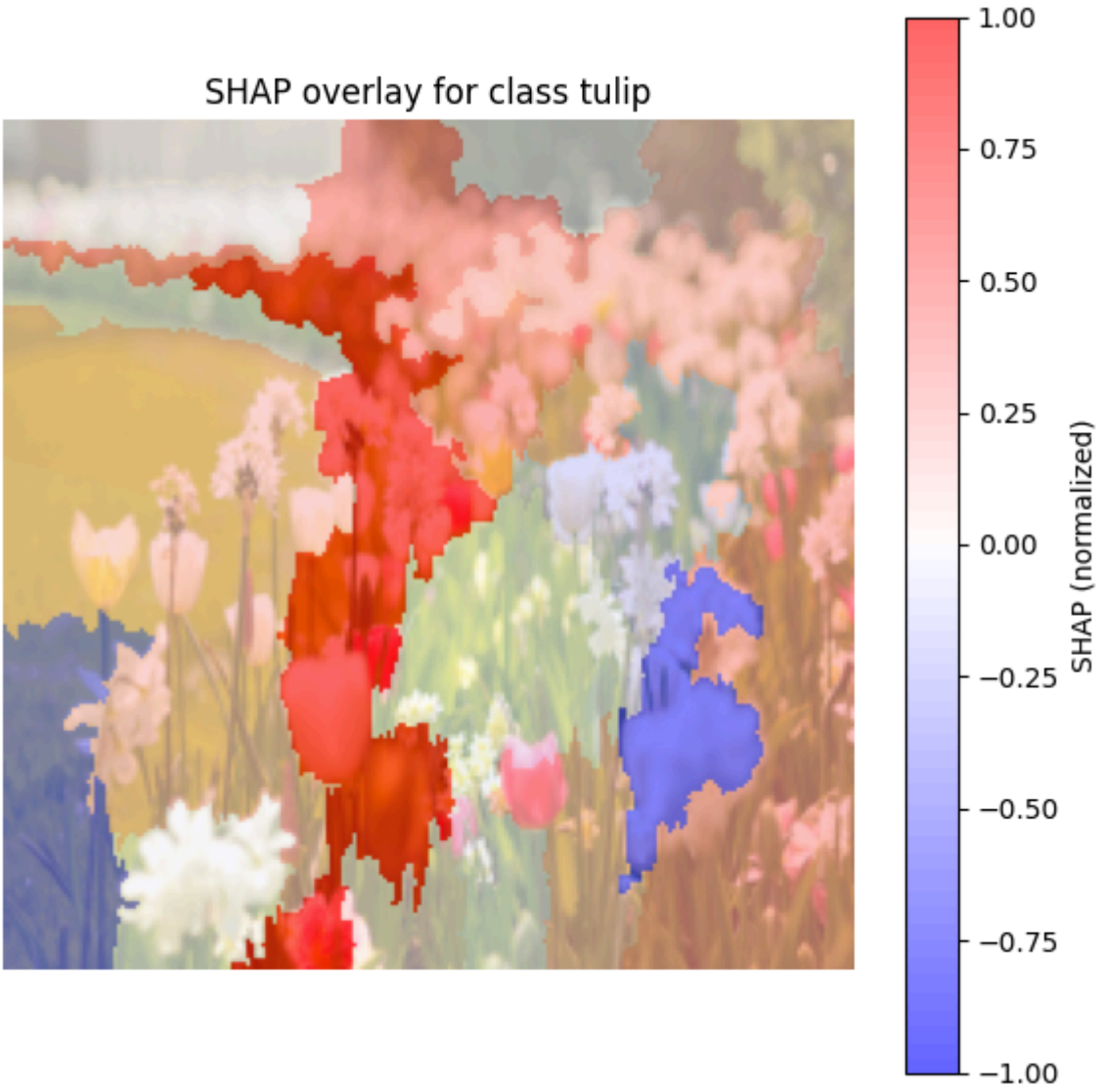
In the rose image, the SHAP analysis shows that the model focused heavily on the **dense, swirling pattern of the rose petals**. The brightest **red regions** are on the petals that are most clearly in focus, which is precisely where the defining characteristics of a rose are found. The darker, out-of-focus background leaves are primarily blue, correctly identified as features that are *not* indicative of a rose.

Sunflower



This is a very interesting case! The model's prediction for "sunflower" comes from two main sources. As expected, **bright red superpixels** cover the classic **orange-yellow petals** of the sunflower. However, there is also a very strong red region covering the **face of the person** in the photo. This suggests that the model may have learned a contextual association from the training data—that images containing sunflowers often contain people as well. While it correctly identified the flower, it also used the person's face as strong evidence for its prediction.

Tulip



For the tulip image, the model's decision was driven by the **classic cup-like shape** of the tulip flowers. The **red areas** predominantly cover the petals of the tulips that are in the center and in focus. The whitish flower in the foreground and some of the background greenery are colored blue, meaning the model recognized them as features that are *not* tulips and correctly discounted their importance.

Key Takeaways

This project provides important insights into the process of building, evaluating, and interpreting a deep learning model for image classification.

- **Effectiveness of Transfer Learning:** The fine-tuning approach, which involved using a pre-trained VGG-16 model with frozen feature-extraction layers, proved to be a highly effective strategy. It enabled the model to achieve a high test accuracy of approximately 90-92% without requiring an exhaustive training process from scratch.

- **Model Validation with Interpretable ML:** By implementing SHAP, we were able to validate that the model learned relevant features for classification. For most classes, the analysis showed that the model correctly focused on key floral characteristics, such as the petals and centers of the daisies and roses, the cup shape of the tulips, and the distinct seed head of the dandelion. The **red superpixels in the SHAP overlays, representing strong positive contributions**, consistently highlighted these correct features.

Github Link for the project : https://github.com/aprameyo8858/Flowers